编译原理作业：第五章 & 第六章

提交方式：提交 PDF 文件至下方链接

```
https://table.nju.edu.cn/dtable/forms/82709267-f336-41e3-9cd2-e6cff0ba3980/
```

截止时间：不设时限，期末考试前提交即可

---

1 第五章

产生式	语义规则
1) $L \to E\ \mathbf{n}$	$L.val = E.val$
2) $E \to E_1\ +\ T$	$E.val = E_1.val + T.val$
3) $E \to T$	$E.val = T.val$
4) $T \to T_1\ *\ F$	$T.val = T_1.val \times F.val$
5) $T \to F$	$T.val = F.val$
6) $F \to (\ E\ )$	$F.val = E.val$
7) $F \to \mathbf{digit}$	$F.val = \mathbf{digit}.\text{lexval}$

图 1: 一个简单的桌上计算器的语法制导定义

产生式	语义规则
1) $T \to F\ T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \to *\ F\ T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \to \epsilon$	$T'.syn = T'.inh$
4) $F \to \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$

图 2: 一个基于适用于自顶向下语法分析文法的 SDD

问题 1. (原书 5.1.2，薄书 5.1.2) 拓展图 2中的 SDD，使它可以像图 1所示的那样处理表达式。

一个更准确的表述为，依照图 1，为图 2制造一个适用于自顶向下语法分析的 SDD，你可能需要先为文法消除左递归。

	产生式	语义规则
1)	$D \to T\ L$	$L.inh = T.type$
2)	$T \to \mathbf{int}$	$T.type = \text{integer}$
3)	$T \to \mathbf{float}$	$T.type = \text{float}$
4)	$L \to L_1\ ,\ \mathbf{id}$	$L_1.inh = L.inh$ $addType(\mathbf{id}.entry, L.inh)$
5)	$L \to \mathbf{id}$	$addType(\mathbf{id}.entry, L.inh)$

图 3: 简单类型声明的语法制导定义

问题 2. (原书 5.2.2，薄书 5.2.2) 考虑图 3中的 SDD，给出下列表达式对应的标注语法分析树

int a, b, c

问题 3. (原书 5.4.3，薄书 5.4.3) 下面的 SDT 计算了一个由 0 和 1 组成的串的值。它把输入的符号串当作正二进制数来解释

$$\begin{aligned} B \ \to\ & B_1\, 0\{B.\text{val} = 2 \times B_1.\text{val}\} \\ |\ & B_1\, 1\{B.\text{val} = 2 \times B_1.\text{val} + 1\} \\ |\ & 1\{B.\text{val} = 1\} \end{aligned}$$

改写这个 SDT，使得基础文法不再是左递归的，但仍然可以计算出整个输入串的相同的 B.val 的值

问题 4. (原书 5.4.4，薄书 5.4.4) 仿照书中例 5.19，为下面的产生式写出一个 L 属性的 SDD 并转换为 SDT

$$S \to \mathbf{do}\ S_1\ \mathbf{while}\ (C)$$

2 第六章

问题 5. (原书 6.1.1，薄书 6.1.1) 为下列表达式构造 DAG

$$((x+y)-((x+y)*(x-y)))+((x+y)*(x-y))$$

问题 6. (原书 6.2.2, 薄书 6.2.2) 考虑下列赋值语句

1. $a = b[i] + c[j]$
2. $a[i] = b * c - b * d$

假定每个数组元素占八个存储单元，将赋值语句翻译成

1. 四元式序列
2. 三元式序列

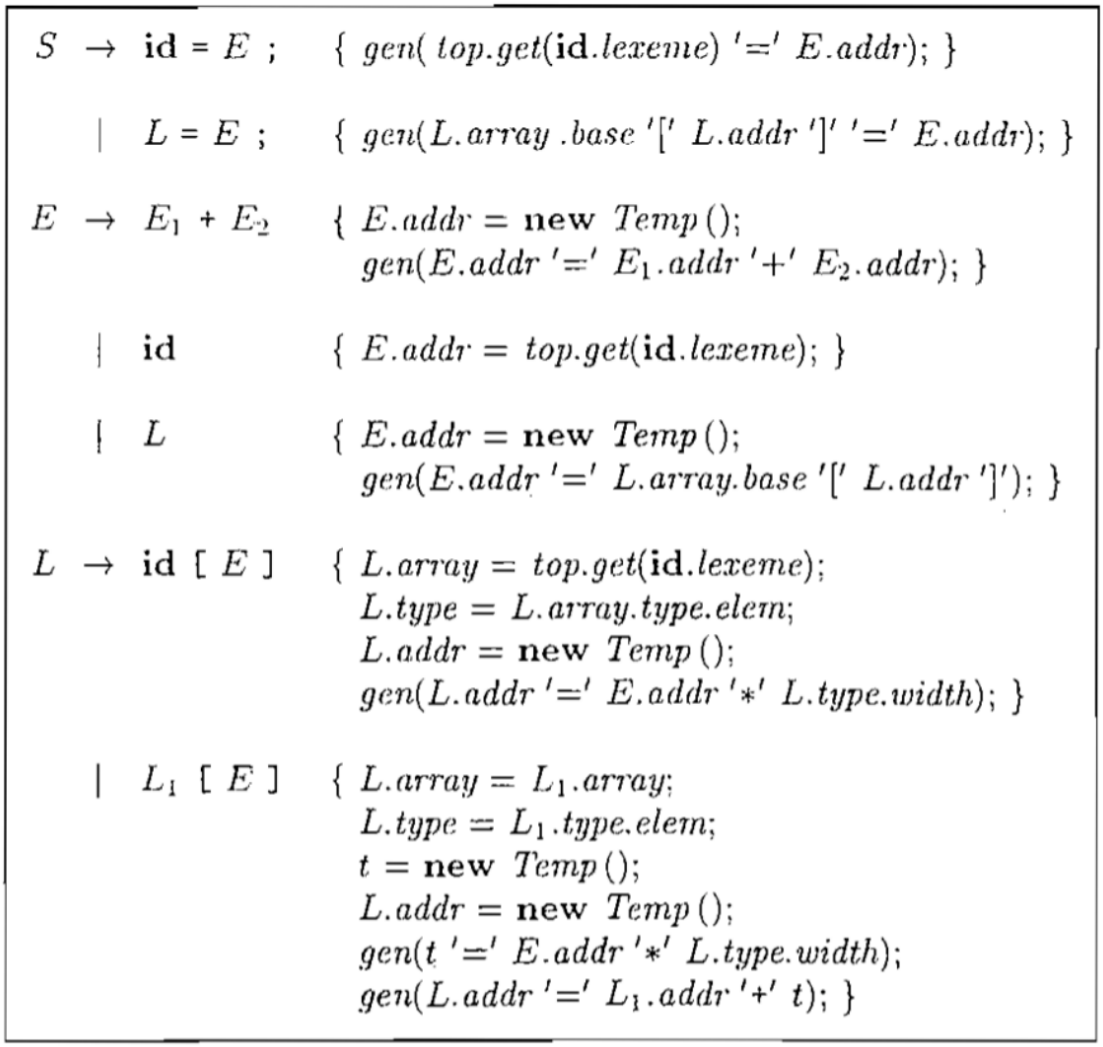


图 4: 处理数组引用的语义动作

问题 7. (原书 6.4.3, 薄书 6.4.3) 使用图 4所示的翻译方案翻译下列赋值语句

$$x = a[i][j] + b[i][j]$$

假定:

1. 一个整数的宽度是 4
2. 假定 a 和 b 均为 $n \times 3$ 的整数数组，即 $a[i]$ 与 $b[i]$ 的宽度均为 $3 \times 4 = 12$，注意到 n 的值不重要

问题 8. (原书 6.4.8, 薄书 6.4.8) 一个实数型数组 $A[i,j,k]$ 的下标范围为 $1 \le i \le 4, 0 \le j \le 4, 5 \le k \le 10$。假定每个实数占 8 个字节并且数组 A 从第 0 字节开始存放，计算下列元素的位置

1. $A[3,4,5]$
2. $A[1,2,7]$
3. $A[4,3,9]$

1)	$B \to B_1$ `||` M B_2	{ $backpatch(B_1.falselist, M.instr)$; $B.truelist = merge(B_1.truelist, B_2.truelist)$; $B.falselist = B_2.falselist$; }
2)	$B \to B_1$ && M B_2	{ $backpatch(B_1.truelist, M.instr)$; $B.truelist = B_2.truelist$; $B.falselist = merge(B_1.falselist, B_2.falselist)$; }
3)	$B \to$! B_1	{ $B.truelist = B_1.falselist$; $B.falselist = B_1.truelist$; }
4)	$B \to$ (B_1)	{ $B.truelist = B_1.truelist$; $B.falselist = B_1.falselist$; }
5)	$B \to E_1$ **rel** E_2	{ $B.truelist = makelist(nextinstr)$; $B.falselist = makelist(nextinstr+1)$; $emit('\texttt{if}'\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ '\texttt{goto}\ _')$; $emit('\texttt{goto}\ _')$; }
6)	$B \to$ **true**	{ $B.truelist = makelist(nextinstr)$; $emit('\texttt{goto}\ _')$; }
7)	$B \to$ **false**	{ $B.falselist = makelist(nextinstr)$; $emit('\texttt{goto}\ _')$; }
8)	$M \to \epsilon$	{ $M.instr = nextinstr$; }

图 5: 布尔表达式的翻译方案

问题 9. (原书 6.7.1, 薄书 6.7.1) 使用图 5的翻译方案翻译下列表达式并给出每个子表达式的真假值列表。假设第一条被生成的指令的地址为 100

$$a == b \&\& (c == d || e == f)$$

1) $S \rightarrow \textbf{if}\,(\,B\,)\,M\,S_1$ { $backpatch(B.truelist,\ M.instr)$;
$S.nextlist\ =\ merge(B.falselist,\ S_1.nextlist)$; }

2) $S \rightarrow\ \textbf{if}\,(\,B\,)\,M_1\,S_1\,N\,\textbf{else}\,M_2\,S_2$
{ $backpatch(B.truelist,\ M_1.instr)$;
$backpatch(B.falselist,\ M_2.instr)$;
$temp\ =\ merge(S_1.nextlist,\ N.nextlist)$;
$S.nextlist\ =\ merge(temp,\ S_2.nextlist)$; }

3) $S \rightarrow\ \textbf{while}\,M_1\,(\,B\,)\,M_2\,S_1$
{ $backpatch(S_1.nextlist,\ M_1.instr)$;
$backpatch(B.truelist,\ M_2.instr)$;
$S.nextlist\ =\ B.falselist$;
$emit('\texttt{goto}\ M_1.instr)$; }

4) $S \rightarrow \{\,L\,\}$ { $S.nextlist\ =\ L.nextlist$; }

5) $S \rightarrow A\ ;$ { $S.nextlist\ =\ \textbf{null}$; }

6) $M \rightarrow \epsilon$ { $M.instr\ =\ nextinstr$; }

7) $N \rightarrow \epsilon$ { $N.nextlist\ =\ makelist(nextinstr)$;
$emit('\texttt{goto}\ _')$; }

8) $L \rightarrow L_1\ M\ S$ { $backpatch(L_1.nextlist,\ M.instr)$;
$L.nextlist\ =\ S.nextlist$; }

9) $L \rightarrow S$ { $L.nextlist\ =\ S.nextlist$; }

图 6: 语句的翻译方案

```
while (E1) {
        if (E2)
                while (E3)
                        S1;
        else {
                if (E4)
                        S2;
                S3
        }
}
```

图 7: 一个程序的控制流结构

问题 10. (原书 6.7.3, 薄书 6.7.3) 当我们使用图 6的规则翻译图 7中的程序时，我们为每个语句 S 生成 S.nextlist。除了图中说明的语句 S_1, S_2, S_3 之外，我们还有另外五个语句结构:

S_4. **while** (E_3) S_1

S_5. **if** (E_4) S_2

S_6. 由 S_5 和 S_3 组合的块

S_7. **if** (E_2) S_4 N **else** S_6

S_8. 整个程序

为每个块 S_i 构造 S_i.next，你可以使用子语句 S_j 的 nextlist、以及程序中任意表达式 E_k 的 E_k.true 和 E_k.false。

a) S_4.next

b) S_5.next

c) S_6.next

d) S_7.next

e) S_8.next

(提示: 我们直接给出最麻烦的情况 S_7 的答案以供参考: S_7.next $= merge(merge(S_4$.nextlist, N.nextlist), S_6.nextlist)，因为它还依赖于一个隐含的节点 N。这是个简单的习题，你的答案应该尽量与实际生成代码时的行为保持一致)